



Date: __ / __ / __

Experiment 3

3. a. Write a C program that utilizes the array data structure to determine whether a given undirected graph is connected or not using the Depth-First Search (DFS) method. The program should take input for the number of vertices, the number of edges, and the edges themselves. It should then use DFS to traverse the graph and output whether the graph is connected or not.

Program

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_VERTICES 100

// Function to initialize the graph
void initializeGraph(int vertices, int
adjacencyMatrix[MAX_VERTICES][MAX_VERTICES]) {
    // Initialize the adjacency matrix with zeros
    for (int i = 0; i < vertices; ++i) {
        for (int j = 0; j < vertices; ++j) {
            adjacencyMatrix[i][j] = 0;
        }
    }
}

// Function to add an edge to the graph
void addEdge(int adjacencyMatrix[MAX_VERTICES][MAX_VERTICES], int start, int end)
{
    adjacencyMatrix[start][end] = 1;
    adjacencyMatrix[end][start] = 1; // For undirected graph
}

// Depth-First Search (DFS) function
void DFS(int vertices, int adjacencyMatrix[MAX_VERTICES][MAX_VERTICES], int
vertex, bool visited[]) {
    visited[vertex] = true;

    for (int i = 0; i < vertices; ++i) {
        if (adjacencyMatrix[vertex][i] && !visited[i]) {
            DFS(vertices, adjacencyMatrix, i, visited);
        }
    }
}
```



```
// Function to check if the graph is connected using DFS
bool isConnected(int vertices, int adjacencyMatrix[MAX_VERTICES][MAX_VERTICES])
{
    bool visited[MAX_VERTICES] = {false};

    // Start DFS from the first vertex
    DFS(vertices, adjacencyMatrix, 0, visited);

    // Check if all vertices are visited
    for (int i = 0; i < vertices; ++i) {
        if (!visited[i]) {
            return false; // Graph is not connected
        }
    }

    return true; // Graph is connected
}

int main() {
    int vertices, edges, start, end;
    int adjacencyMatrix[MAX_VERTICES][MAX_VERTICES];

    printf("Enter the number of vertices in the graph: ");
    scanf("%d", &vertices);

    initializeGraph(vertices, adjacencyMatrix);

    printf("Enter the number of edges in the graph: ");
    scanf("%d", &edges);

    printf("Enter the edges (start end):\n");
    printf("Use starting vertex ID as 0 \n");
    for (int i = 0; i < edges; ++i) {
        scanf("%d %d", &start, &end);
        addEdge(adjacencyMatrix, start, end);
    }

    if (isConnected(vertices, adjacencyMatrix)) {
        printf("The graph is connected.\n");
    } else {
        printf("The graph is not connected.\n");
    }

    return 0;
}
```



Sample Output

Enter the number of vertices in the graph: 4

Enter the number of edges in the graph: 6

Enter the edges (start end):

Use starting vertex ID as 0

0 1

1 2

2 3

3 0

0 3

1 2

The graph is connected.

Enter the number of vertices in the graph: 4

Enter the number of edges in the graph: 4

Enter the edges (start end):

Use starting vertex ID as 0

0 1

1 2

2 0

0 1

The graph is not connected.